Project 2: MLP Implementation

Objectives

Your goal in this project is to better understand how forward and backward passes work in a multi-layer perceptron:

Deliverable

- Project report/writeup: A `project2_report_lastname.pdf` file generated from the below document.

  The project report should be generated by, following completion of the assignment, printing the resulting document to a pdf file and submitting that file. The document should additionally include a brief justification of your solution at a high-level, e.g., using any relevant explanations, equations, or pictures that help to explain your solution. You should **answer each question in-line**, and all questions we ask you to answer must be present.. You should also **describe what your code does**, e.g. using a couple of sentences per function to describe your code structure, in a section following the code itself. The objective is to make the report self-contained for grading, while ensuring that you wrote the code yourself.

    - To be specific, this means your report will include (but not be limited to):
        1. Any code snippets you implemented
        2. Plots affected by your implementation, e.g., scatter plots with decision boundaries or loss curves.
        3. Output results of the questions, e.g., the best lambda from cross validation.
        4. Please do NOT include debugging messages.

Logistics

- You complete and submit the project report/writeup individually. While you may discuss the core concepts, the logic, and the assigned questions with others, the source code and project report needs to be written independently.

- Source code should be left in-line below. Failure to provide source code will lead to a deduction of points from your total.

In [ ]:
```
# install seaborn for better visualization
!pip3 install scikit-learn numpy matplotlib seaborn
```

Task 1: Forward and Backward Propagation (40 points)

Your goal in this exercise is to classify handwritten digits with neural networks. The training and test units, together with the structure of a multilayer perceptron class are already provided in the starter code. Your task is to code up the key lines that complete the **forward** and **backward** propagation functions.

forward: Recall that forward propagation corresponds to the prediction algorithm for neural nets. The intermediate outputs for this method will be cached for backpropagation during the training phase.

backward: Backpropagation collects all the gradients necessary for training the neural network, which will be used as input for the minibatch SGD algorithm GradientDescentOptimizer

In the next, you will need to implement the forward pass and backward pass of the linear layer. For the forward pass, it takes x_l as input and outputs x_{l+1}.

$$x_{l+1} = w^\top x_{l} + b_{l}$$

For the backward pass, you will need to compute the backward gradients,

$$\frac{\partial \ell(x, y)}{\partial w}=?$$ $$\frac{\partial \ell(x, y)}{\partial b}=?$$ $$\frac{\partial \ell(x, y)}{\partial x_{l}}=?$$

where ℓ is the loss function.

In [7]:
```
import numpy as np

class Layer(object):
    def __init__(self, shape, activ_func):
        "Implements a layer of a NN."

        self.w = np.random.uniform(-np.sqrt(2.0 / shape[0]),
                                   np.sqrt(2.0 / shape[0]),
                                   size=shape)
        self.b = np.zeros((1, shape[1]))

        # The activation function, for example, RELU, tanh, or sigmoid.
```

```
        self.activate = activ_func

        # The derivative of the activation function.
        self.d_activate = GRAD_DICT[activ_func]

    def forward(self, inputs):
        """Forward propagate the activation through the layer.

        Given the inputs (activation of previous layers),
        compute and save the activation of current layer,
        then return it as output.
        """

        ####################################Description begins#################
        # Forward pass

        # Use the linear and non-linear transformation to
        # compute the activation and cache it in a the field, self.a.

        # Functions you may use:
        # np.dot: numpy function to compute dot product of two matrix.
        # self.activate: the activation function of this layer,
        #                it takes in a matrix of scores (linear transformation
        #                and compute the activations (non-linear transformatic
        # (plus the common arithmetic functions).

        # For all the numpy functions, use google and numpy manual for
        # more details and examples.

        # Object fields you will use:
        # self.w:
        #     weight matrix, a matrix with shape (H_-1, H).
        #     H_-1 is the number of hidden units in previous layer
        #     H is the number of hidden units in this layer
        # self.b: bias, a matrix/vector with shape (1, H).
        # self.activate: the activation function of this layer.

        # Input:
        # inputs:
        #    a matrix with shape (N, H_-1),
        #    N is the number of data points.
        #    H_-1 is the number of hidden units in previous layer

        ####################################Description ends###################
        # Modify the right hand side of the following code.

        # The linear transformation.
        # scores:
        #     weighted sum of inputs plus bias, a matrix of shape (N, H).
        #     N is the number of data points.
        #     H is the number of hidden units in this layer.

        scores = np.dot(inputs, self.w) + self.b ## Your code here ##
```

```
        # End of your code


        # The non-linear transformation.
        # outputs:
        #     activations of this layer, a matrix of shape (N, H).
        #     N is the number of data points.
        #     H is the number of hidden units in this layer.

        activations = self.activate(scores) ## Your code here ##
        # End of your code

        # End of the code to modify
        ##########################################################

        # Cache the inputs and the activations (to be used by backprop).

        self.inputs = inputs
        self.a = activations
        outputs = activations
        return outputs

    def backward(self, d_outputs):
        """Backward propagate the gradient through this layer.

        Given the gradient w.r.t the output of this layer
        (d_outputs), compute and save the gradient w.r.t the
        weights (d_w) and bias (d_b) of this layer and
        return the gradient w.r.t the inputs (d_inputs).
        """
        ###################################Description begins#################
        # Backpropagation

        # Compute the derivatives of the loss w.r.t the weights and bias
        # given the derivatives of the loss w.r.t the outputs of this layer
        # using chain rule.

        # Naming convention: use d_var to store the
        # derivative of the loss w.r.t the variable.

        # Functions you may use:
        # np.dot (numpy.dot): numpy function to compute dot product of two mat
        # np.mean or np.sum (numpy.mean or numpy.sum):
        #     numpy function to compute the mean or sum of a matrix,
        #     use keywords argument 'axis' to compute the mean
        #     or sum along a particular axis, you might also
        #     found 'keepdims' argument useful.
        # self.d_activate:
        #     given the current activation (self.a) as input,
        #     compute the derivative of the activation function,
        #     See d_relu as an example.
        # (plus the common arithmetic functions).
        # np.transpose or m.T (m is an numpy array): transpose a matrix.
```

```
        # Object fields you will use:
        # self.w: weight matrix, a matrix with shape (H_-1, H).
        #         H_-1 is the number of hidden units in previous layer
        #         H is the number of hidden units in this layer
        # self.d_activate: compute derivative of the activation function.
        #                  See d_relu as an example.
        # d_outputs: the derivative of the loss w.r.t the outputs of
        #            this layer, a matrix of shape (N, H). N is the number of
        #            data points and H is the number of hidden units in this l
        # self.inputs: inputs to this layer, a matrix with shape (N, H_-1)
        #              N is the number of data points.
        #              H_-1 is the number of hidden units in previous layer.
        # self.a: activation of the hidden units of this layer, a matrix
        #         with shape (N, H)
        #         N is the number of data points.
        #         H is the number of hidden units in this layer.

        ###################################Description ends###################


        # Modify the right hand side of the following code.

        # d_scores:
        #     Derivatives of the loss w.r.t the scores (the result from linear
        #     A matrix of shape (N, H)
        #     N is the number of data points.
        #     H is the number of hidden units in this layer.
        d_scores = d_outputs * self.d_activate(self.a) ## Your code here ##
        # End of your code

        # self.d_b:
        #     Derivatives of the loss w.r.t the bias, summed over all data poi
        #     A matrix of shape (1, H)
        #     H is the number of hidden units in this layer.

        self.d_b = np.sum(d_scores, axis=0, keepdims=True) ## Your code here #
        # End of your code

        # self.d_w:
        #     Derivatives of the loss w.r.t the weight matrix, summed over all
        #     A matrix of shape (H_-1, H)
        #     H_-1 is the number of hidden units in previous layer
        #     H is the number of hidden units in this layer.

        self.d_w = np.dot(self.inputs.T, d_scores) ## Your code here ##
        # End of your code

        # d_inputs:
        #     Derivatives of the loss w.r.t the previous layer's activations/o
        #     A matrix of shape (N, H_-1)
        #     N is the number of data points.
```

```
        #     H_-1 is the number of hidden units in the previous layer.

        d_inputs = np.dot(d_scores, self.w.T) ## Your code here ##
        # End of your code

        # End of the code to modify
        ##########################################################

                # Compute the average value of the gradients, since
        # we are minimizing the average loss.
        self.d_b /= d_scores.shape[0]
        self.d_w /= d_scores.shape[0]

        return d_inputs
```

You have successfully completed task 1. Please run the code to define the Multi-Layer Perceptron (MLP).

In [8]:

```
import numpy as np

########################
#  Multilayer Perceptron
########################

class MLP(object):
    def __init__(self, input_dim, output_dim, sizes, activ_funcs):
        """Multilayer perceptron for multi-class classification.

        The object holds a list of layer objects, each one
        implements a layer in the network, the specification
        of each layer is decided by input_dim, output_dim,
        sizes and activ_funcs. Note that an output layer
        (linear) and loss function (softmax and
        cross-entropy) would be automatically added to the MLP.

        Input:
          input_dim: dimension of input.
          output_dim: dimension of output (number of labels).
          sizes: a list of integers specifying the number of
            hidden units on each layer.
          activ_funcs: a list of function objects specifying
            the activation function of each layer.

        """
        # Last layer is linear and loss is mean_cross_entropy_softmax
        self.sizes = [input_dim] + sizes[:] + [output_dim]
        self.activ_funcs = activ_funcs[:] + [linear]
        self.shapes = []
        for i in range(len(self.sizes)-1):
            self.shapes.append((self.sizes[i], self.sizes[i+1]))

        self.layers = []
        for i, shape in enumerate(self.shapes):
```

```
            self.layers.append(Layer(shape, self.activ_funcs[i]))

    def forwardprop(self, data, labels=None):
        """Forward propagate the activations through the network.

        Iteratively propagate the activations (starting from
        input data) through each layer, and output a
        probability distribution among labels (probs), and
        if labels are given, also compute the loss.
        """
        inputs = data
        for layer in self.layers:
            outputs = layer.forward(inputs)
            inputs = outputs

        probs = softmax(outputs)
        if labels is not None:
            return probs, self.loss(outputs, labels)
        else:
            return probs, None

    def backprop(self, labels):
        """Backward propagate the gradients/derivatives through the network.

        Iteratively propagate the gradients/derivatives (starting from
        outputs) through each layer, and save gradients/derivatives of
        each parameter (weights or bias) in the layer.
        """
        d_outputs = self.d_loss(self.layers[-1].a, labels)
        for layer in self.layers[::-1]:
            d_inputs = layer.backward(d_outputs)
            d_outputs = d_inputs

    def loss(self, outputs, labels):
        "Compute the cross entropy softmax loss."
        return mean_cross_entropy_softmax(outputs, labels)

    def d_loss(self, outputs, labels):
        "Compute derivatives of the cross entropy softmax loss w.r.t the outpu
        return d_mean_cross_entropy_softmax(outputs, labels)

    def predict(self, data):
        "Predict the labels of the data."
        probs, _ = self.forwardprop(data)
        return np.argmax(probs, axis=1)


class GradientDescentOptimizer(object):
    def __init__(self, learning_rate, decay_steps=1000,
                 decay_rate=1.0):
        "Gradient descent with staircase exponential decay."
        self.learning_rate = learning_rate
        self.steps = 0.0
```

```
        self.decay_steps = decay_steps
        self.decay_rate = decay_rate

    def update(self, model):
        "Update model parameters."
        for layer in model.layers:
            layer.w -= layer.d_w * self.learning_rate
            layer.b -= layer.d_b * self.learning_rate
        self.steps += 1
        if (self.steps + 1) % self.decay_steps == 0:
            self.learning_rate *= self.decay_rate


# Utility functions.
def sigmoid(x):
    return 1/(1+np.exp(-x))

def d_sigmoid(a=None, x=None):
    if a is not None:
        return a * (1 - a)
    else:
        return d_sigmoid(a=sigmoid(x))

def relu(x):
    "The rectified linear activation function."
    return np.clip(x, 0.0, None)

def d_relu(a=None, x=None):
    "Compute the derivative of RELU given activation (a) or input (x)."
    if a is not None:
        d = np.zeros_like(a)
        d[np.where(a > 0.0)] = 1.0
        return d
    else:
        return d_relu(a=relu(x))

def tanh(x):
    "The tanh activation function."
    return np.tanh(x)

def d_tanh(a=None, x=None):
    "The derivative of the tanh function."
    if a is not None:
        return 1 - a ** 2
    else:
        return d_tanh(a=tanh(x))

def linear(x):
    return x

def d_linear(a=None, x=None):
    return 1.0
```

```
def softmax(x):
    shifted_x = x - np.max(x, axis=1, keepdims=True)
    f = np.exp(shifted_x)
    p = f / np.sum(f, axis=1, keepdims=True)
    return p

def mean_cross_entropy(outputs, labels):
    n = labels.shape[0]
    return - np.sum(labels * np.log(outputs)) / n

def mean_cross_entropy_softmax(logits, labels):
    return mean_cross_entropy(softmax(logits), labels)

def d_mean_cross_entropy_softmax(logits, labels):
    return softmax(logits) - labels


# Mapping from activation functions to its derivatives.
GRAD_DICT = {linear: d_linear, sigmoid: d_sigmoid, tanh: d_tanh, relu: d_relu}
```

Now that we have defined the MLP, let's prepare the data.

We will still use the MNIST dataset, but we will preprocess the data differently for the neural network.

We will normalize the pixel values to be between 0 and 1 and use one-hot encoding for the labels.

In [9]:
```
# Download dataset
from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split

print('Loading and preprocessing data')
# Load data from https://www.openml.org/d/554
X, y = fetch_openml("mnist_784", version=1, return_X_y=True, as_frame=False, d
X = X / 255.0
y = y.astype(int)

(train_data, test_data, train_labels, test_labels) = train_test_split(X, y, ra

def create_one_hot_labels(labels, dim=10):
    one_hot_labels = np.zeros((labels.shape[0], dim))
    for i in range(labels.shape[0]):
        one_hot_labels[i][labels[i]] = 1
    return one_hot_labels

# Convert labels from integers to one-hot encodings
test_labels = create_one_hot_labels(test_labels)
train_labels = create_one_hot_labels(train_labels)
```

```
Loading and preprocessing data
```

Explanation of the Code

Explain how the code ***you wrote*** the above sections works, using a couple of sentences per function to describe your code structure and any choices you made in its implementation.

ANSWER: Forward method:

- To calculate scores, I used the variables defined in the function description to carry out the Z = X dot W + B operation (weighted sum + bias). This transforms the data (the inputs) into the current layers' feature space. In code, the X dot W operation was carried out using numpys np.dot function. This created the weighted sum. Because the resulting matrix of this operation was an N x H matrix, and self.b is a (1, H) we can simply use numpy to add self.b to it (element-wise addition) to get our final result: scores (Z) (N x H matrix).
- The second line of code I implemented was applying the layer's stored activation function (described in the description as self.activate) to the scores matrix that we calculated above. self.activate is an element-wise operation that turns Z (scores) into A (activations). This is an important step that allows our neural network to model nonlinear relationships in the data - we are introducing nonlinearity with the activation function.

Backward method: Overall, this method calculated the gradients of the loss function with respect to Z (scores), W (self.w), B (self.b), and X (inputs).

- The first step is to calculate d_scores, the derivative of scores, by applying the Chain Rule (as described in the description) across the activation function. Here I am converting the gradient of the loss function with respect to the activations we calculated in Forward, into the gradient with respect to the linear scores (Z). This is simply element-wise multiplication, and both d_outputs and self.d_activate(self.a) are the same dimensions, so the resulting dZ also has shape (N x H).
- Next, we calculate self.d_b by summing d_scores (collapsing the N dimension with axis=0 and outputting a (1 x H) result with keepdims = True). We can just sum d_scores because the bias terms are added to every sample's score, so the gradient with respect to the bias is just the sum of the gradient with respect to scores (dZ) across all inputs.
- Then we calculate self.d_w, which calculates the total gradient of the loss with respect to the weights (W) for all inputs. This is the dot product of the transposed inputs (X^T) and the scores gradient (dZ).

We get the transpose (X^T) through self.inputs.T, and dZ was calculated above (d_scores).

- The last piece of code I filled in was calculating d_inputs (dX). Here we are calculating the gradient of the loss with respect to the input of the layer (the output of the previous layer). To do this, we take the dot product of d_scores (dZ) and the transpose of W (self.w.T).

I used Grammarly within Google Docs to correct my spelling and grammar since typing in VS Code doesn't have any spell check.

Task 2 Training the Neural Network (25pt)

Report the training loss, training accuracy, test loss, and test accuracy for the default network architecture.

` MLP(784, 10, [16], [sigmoid])`

In [10]:
```
# Initialize model
print('Initializing neural network')
model = MLP(784, 10, [16], [sigmoid])

selected = np.random.randint(test_data.shape[0], size=100)
true_labels = np.argmax(test_labels[selected], axis=1)
preds_init = model.predict(test_data[selected])
```

```
Initializing neural network
```

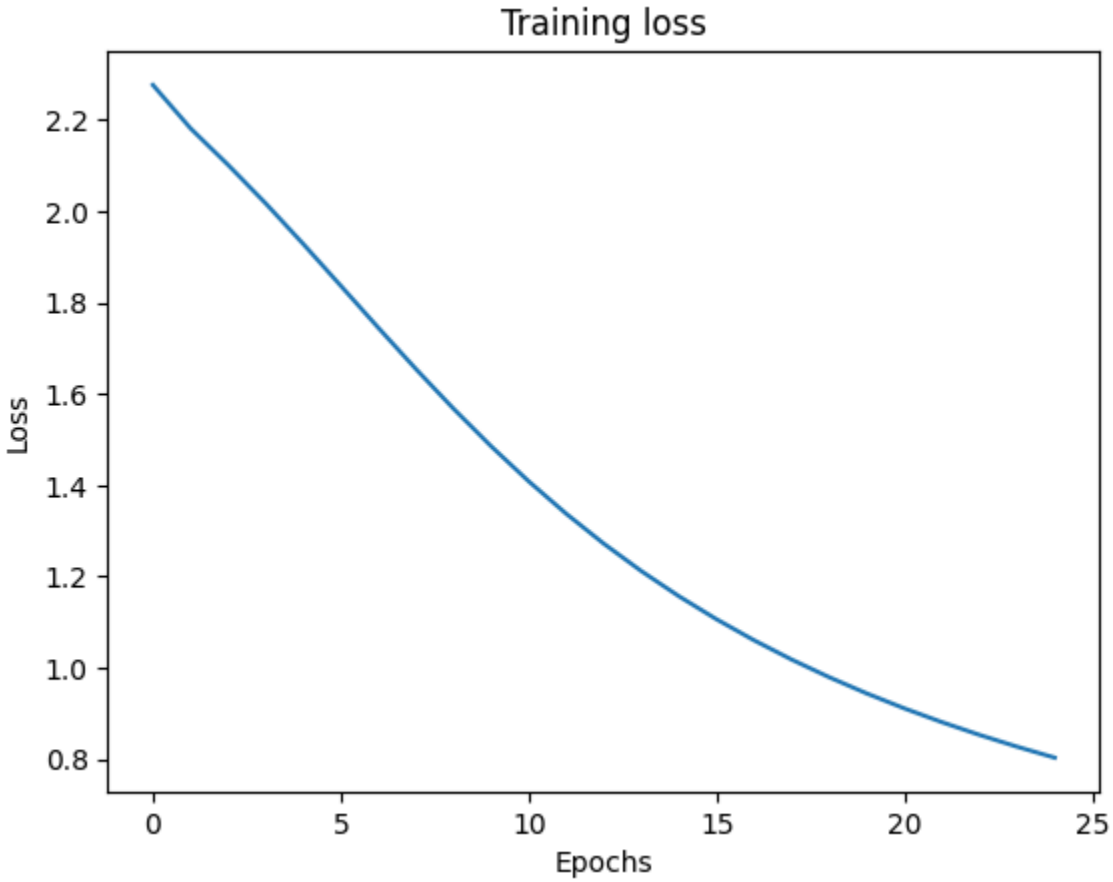
In [11]:
```
print('Start training')

n_train = train_data.shape[0]
n_epochs = 25
batch_size = 100
opt = GradientDescentOptimizer(0.01)

train_loss_list = []
train_accuracy_list = []
for i in range(n_epochs):
    sum_loss = 0.0
    for j in range((n_train - 1) // batch_size + 1):
        batch_data = train_data[j*batch_size:(j+1)*batch_size]
        batch_labels = train_labels[j*batch_size:(j+1)*batch_size]
        _, loss = model.forwardprop(batch_data, batch_labels)
        if np.isnan(loss):
            print('batch %s loss is abnormal')
            print(loss)
            continue
        sum_loss += loss
        model.backprop(batch_labels)
        opt.update(model)
```

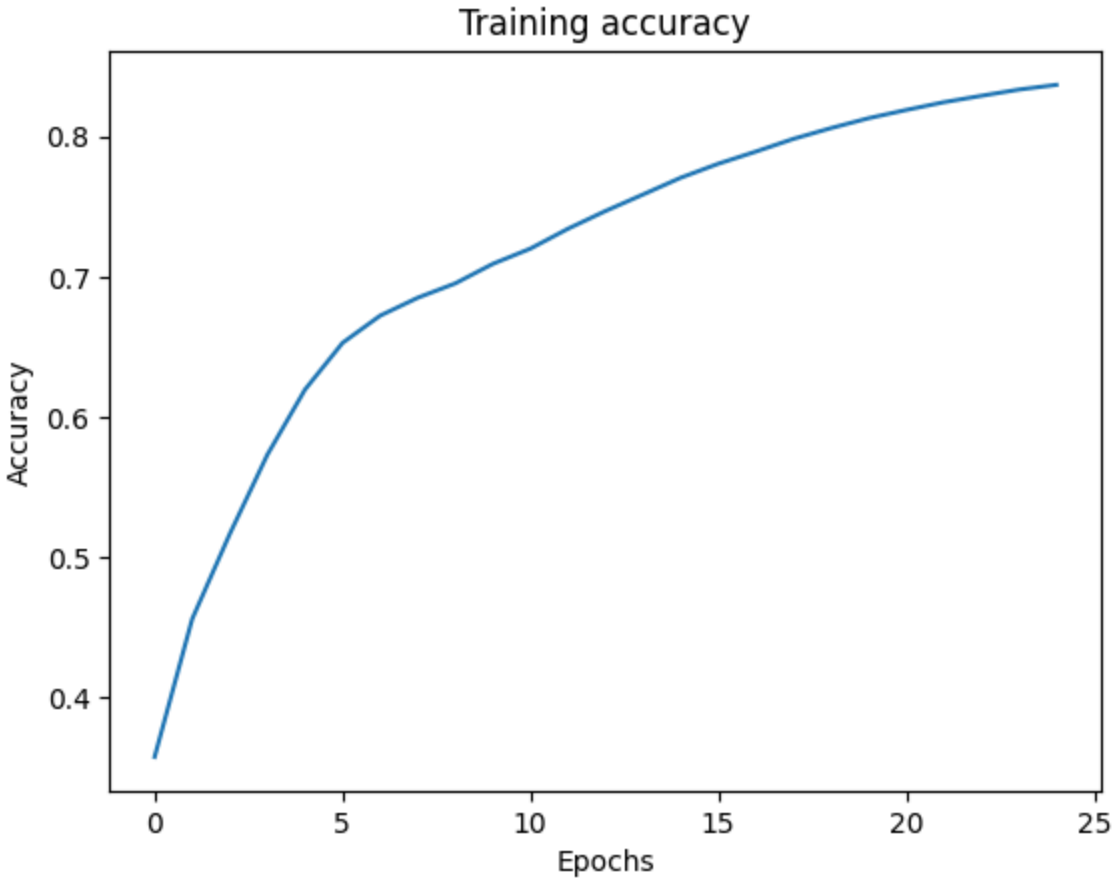
```
    train_loss = sum_loss/(j+1)
    train_accuracy = (np.sum(model.predict(train_data) ==
                             np.argmax(train_labels, axis=1)) /
                      np.float64(train_labels.shape[0]))
    train_loss_list.append(train_loss)
    train_accuracy_list.append(train_accuracy)
    print('=' * 20 + ('Epoch %d' % i) + '=' * 20)
    print('Train loss %s accuracy %s' % (train_loss, train_accuracy))

# Compute test loss and accuracy.
_, test_loss = model.forwardprop(test_data, test_labels)
test_accuracy = (np.sum(model.predict(test_data) ==
                        np.argmax(test_labels, axis=1)) /
                 np.float64(test_labels.shape[0]))
print('=' * 20 + 'Training finished' + '=' * 20 + '\n')
print ('Test loss %s accuracy %s\n' %
        (test_loss, test_accuracy))

preds_trained = model.predict(test_data[selected])
```

```
Start training
====================Epoch 0====================
Train loss 2.27659836298836 accuracy 0.3580952380952381
====================Epoch 1====================
Train loss 2.1817271786012093 accuracy 0.4562857142857143
====================Epoch 2====================
Train loss 2.1015823839108725 accuracy 0.5168571428571429
====================Epoch 3====================
Train loss 2.0173424569690743 accuracy 0.5731904761904761
====================Epoch 4====================
Train loss 1.9284946481384113 accuracy 0.6195238095238095
====================Epoch 5====================
Train loss 1.8368633875760465 accuracy 0.6528095238095238
====================Epoch 6====================
Train loss 1.7447862353580215 accuracy 0.6721904761904762
====================Epoch 7====================
Train loss 1.654446409726421 accuracy 0.684952380952381
====================Epoch 8====================
Train loss 1.5675839105089306 accuracy 0.6951904761904761
====================Epoch 9====================
Train loss 1.4854053578977644 accuracy 0.7090952380952381
====================Epoch 10====================
Train loss 1.4086046360988662 accuracy 0.7199523809523809
====================Epoch 11====================
Train loss 1.3374540884316821 accuracy 0.7340952380952381
====================Epoch 12====================
Train loss 1.271924262011072 accuracy 0.7467619047619047
====================Epoch 13====================
Train loss 1.2117939635985715 accuracy 0.7585238095238095
====================Epoch 14====================
Train loss 1.1567328214740862 accuracy 0.7704285714285715
====================Epoch 15====================
Train loss 1.1063567504172016 accuracy 0.7803809523809524
====================Epoch 16====================
Train loss 1.0602636252116842 accuracy 0.789047619047619
====================Epoch 17====================
Train loss 1.0180559326954495 accuracy 0.7980952380952381
====================Epoch 18====================
Train loss 0.9793547930904116 accuracy 0.8057619047619048
====================Epoch 19====================
Train loss 0.9438079988225374 accuracy 0.8128095238095238
====================Epoch 20====================
Train loss 0.9110938335282399 accuracy 0.8186666666666667
====================Epoch 21====================
Train loss 0.8809220016115742 accuracy 0.8241904761904761
====================Epoch 22====================
Train loss 0.8530327142643945 accuracy 0.8287619047619048
====================Epoch 23====================
Train loss 0.8271947253008965 accuracy 0.8331904761904761
====================Epoch 24====================
Train loss 0.8032028763576978 accuracy 0.8365714285714285
====================Training finished====================

Test loss 0.7905584896142165 accuracy 0.8410408163265306
```

Plot the training loss and accuracy as a function of the number of epochs for debugging.

In [12]:
```
import matplotlib.pyplot as plt

# Plot the training loss and accuracy separately.
plt.plot(range(n_epochs), train_loss_list, label='train loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.title('Training loss')
plt.show()

plt.plot(range(n_epochs), train_accuracy_list, label='train accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.title('Training accuracy')
plt.show()
```



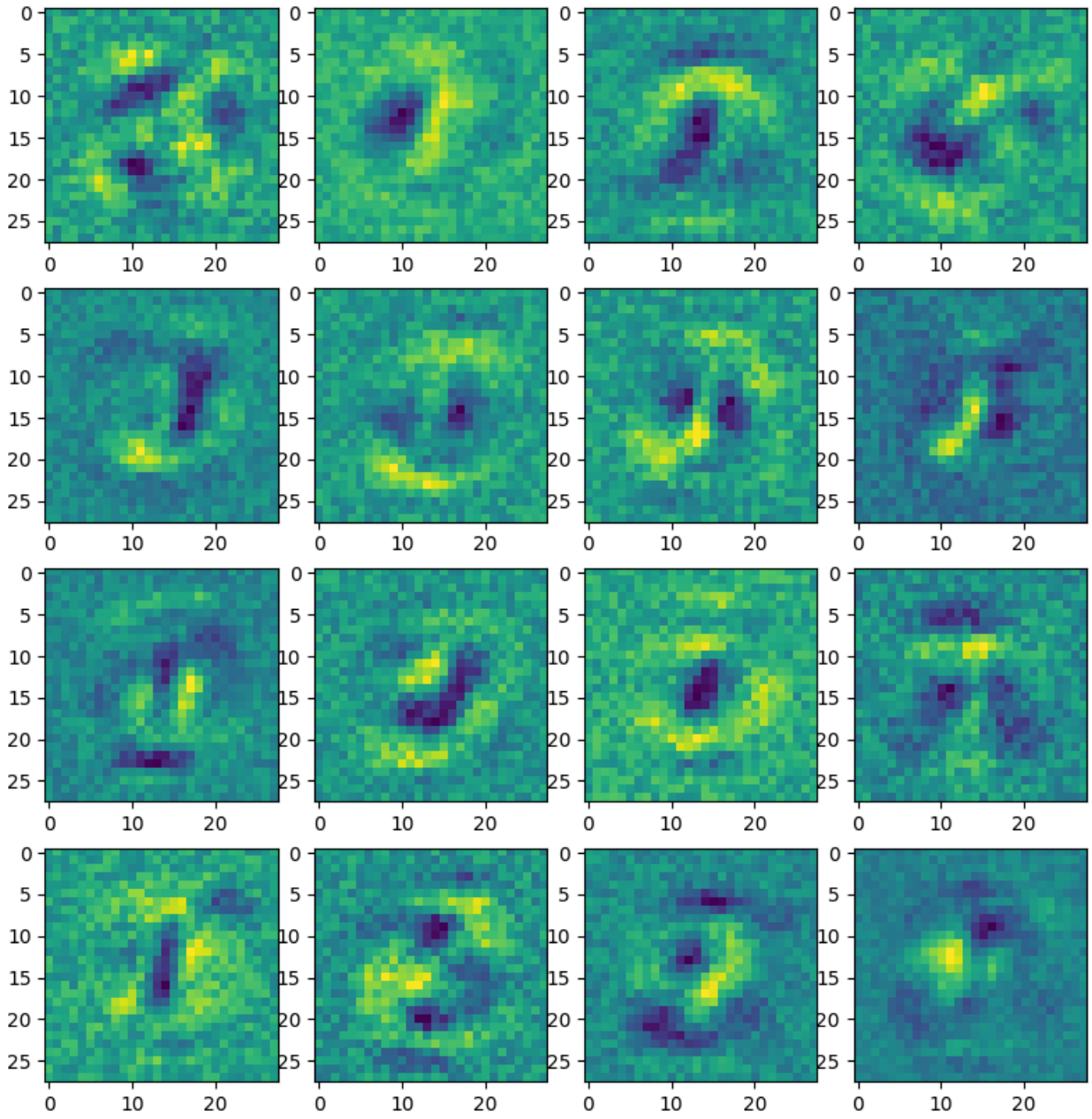


Task 3: Visualizing the Predictions and Weights (10pt)

In your solution, **include a plot of the test samples using the following code.**

In [13]:
```
import matplotlib.pyplot as plt

# plot sample test images together with their groundtruth
# and predicted labels before and after training

fig, axes = plt.subplots(10, 10, figsize=(10, 10))
fig.subplots_adjust(wspace=0)
for a, image, true_label, pred_init, pred_trained in zip(
        axes.flatten(), test_data[selected],
        true_labels, preds_init, preds_trained):
    a.imshow(image.reshape(28, 28), cmap='gray_r')
    a.text(0, 10, str(true_label), color="black", size=15)
    a.text(0, 26, str(pred_trained), color="blue", size=15)
    a.text(22, 26, str(pred_init), color="red", size=15)

    a.set_xticks(())
    a.set_yticks(())

plt.show()
```

In [14]:
```
fig, axes = plt.subplots(4, 4, figsize=(10, 10))
fig.subplots_adjust(wspace=0)
idx = 0
for a in axes.flatten():
    a.imshow(model.layers[0].w[:, idx].reshape((28, 28)))
    idx +=1
plt.show()
```



Task 4: Hyperparameter tuning (15 pts)

Modify only the network architecture (e.g. number of hidden units, activation functions, etc.) to improve the test accuracy, and provide a summary of your findings.

In [ ]:
```
# Initialize model
print('Initializing neural network')

model = MLP(784, 10, [256], [relu]) ## Your code here ## this is my final choi

################## Do not modify the following code ############
print('Start training')
```

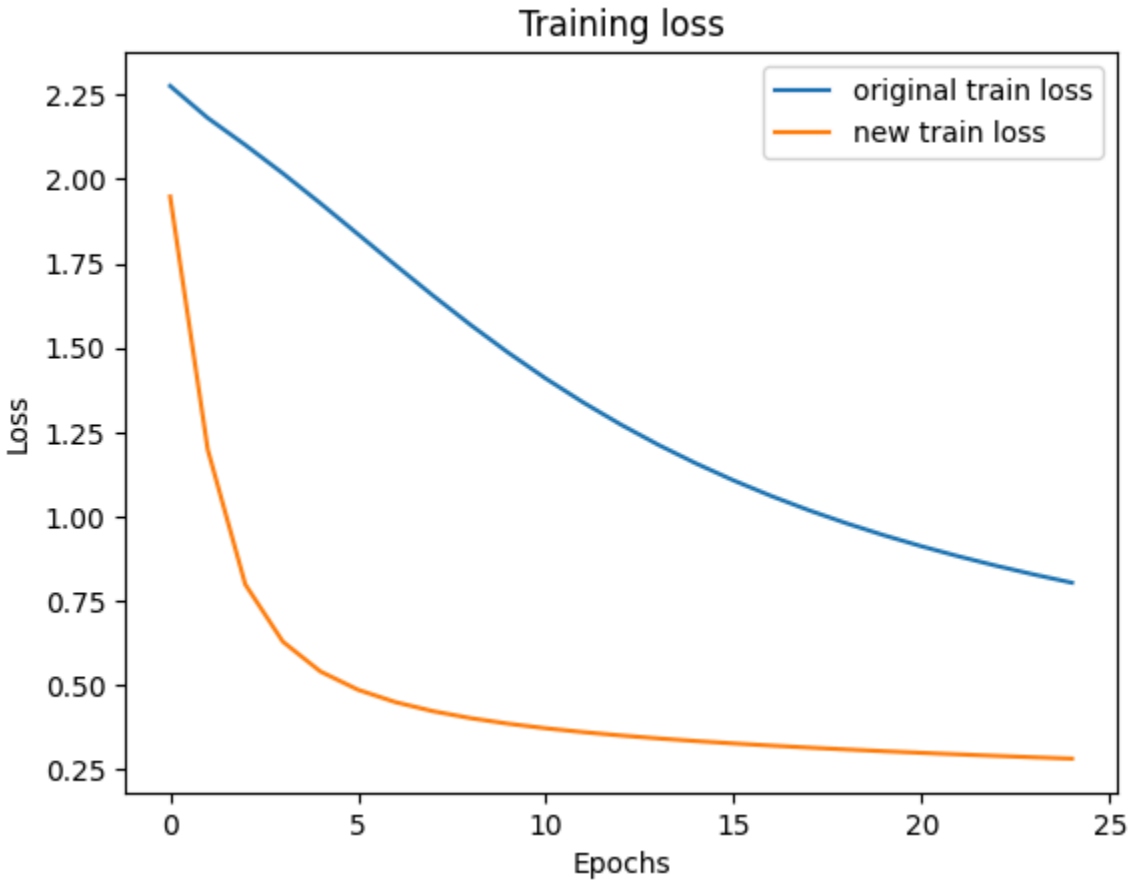
```
n_train = train_data.shape[0]
n_epochs = 25
batch_size = 100
opt = GradientDescentOptimizer(0.01)

part_4_train_loss_list = []
part_4_train_accuracy_list = []
for i in range(n_epochs):
    sum_loss = 0.0
    for j in range((n_train - 1) // batch_size + 1):
        batch_data = train_data[j*batch_size:(j+1)*batch_size]
        batch_labels = train_labels[j*batch_size:(j+1)*batch_size]
        _, loss = model.forwardprop(batch_data, batch_labels)
        if np.isnan(loss):
            print('batch %s loss is abnormal')
            print(loss)
            continue
        sum_loss += loss
        model.backprop(batch_labels)
        opt.update(model)
    train_loss = sum_loss/(j+1)
    train_accuracy = (np.sum(model.predict(train_data) ==
                             np.argmax(train_labels, axis=1)) /
                      np.float64(train_labels.shape[0]))
    part_4_train_loss_list.append(train_loss)
    part_4_train_accuracy_list.append(train_accuracy)
    print('=' * 20 + ('Epoch %d' % i) + '=' * 20)
    print('Train loss %s accuracy %s' % (train_loss, train_accuracy))

# Compute test loss and accuracy.
_, test_loss = model.forwardprop(test_data, test_labels)
test_accuracy = (np.sum(model.predict(test_data) ==
                        np.argmax(test_labels, axis=1)) /
                 np.float64(test_labels.shape[0]))
print('=' * 20 + 'Training finished' + '=' * 20 + '\n')
print ('Test loss %s accuracy %s\n' %
       (test_loss, test_accuracy))

part_4_preds_trained = model.predict(test_data[selected])
```

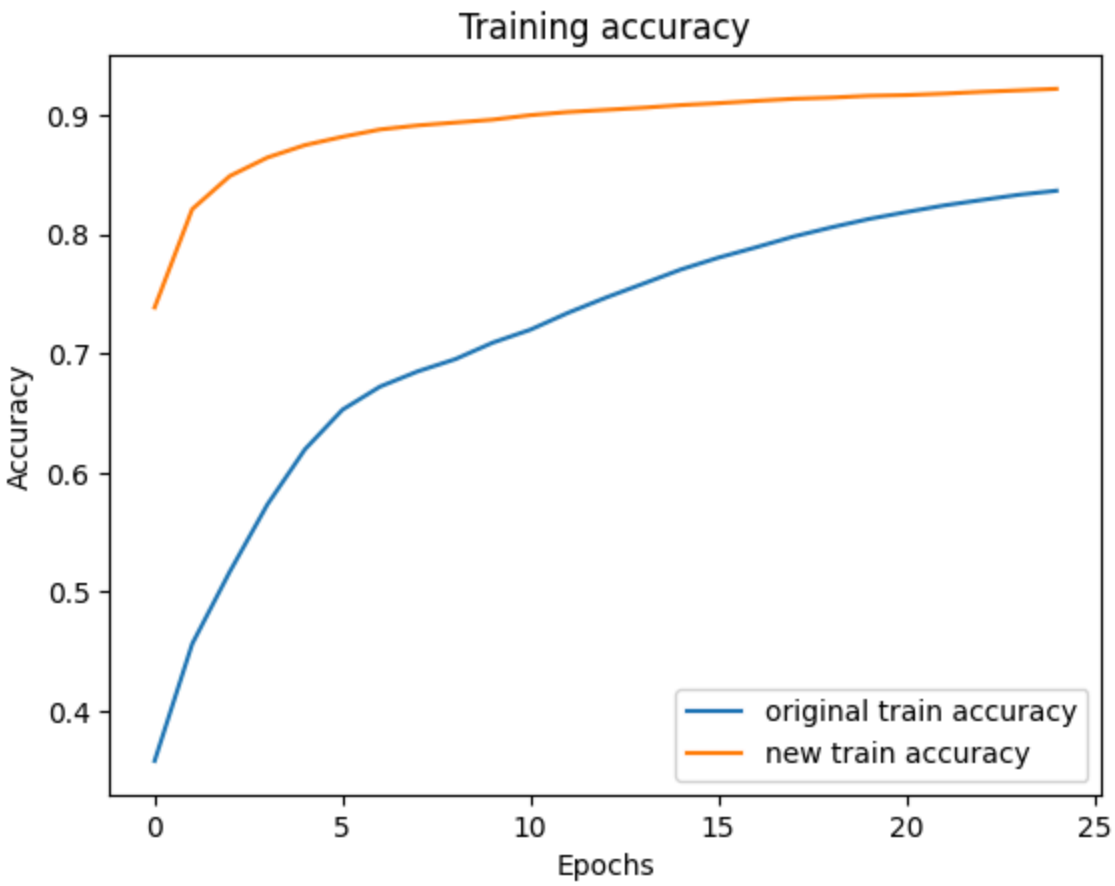
```
Initializing neural network
Start training
====================Epoch 0====================
Train loss 1.9484916679935198 accuracy 0.7386190476190476
====================Epoch 1====================
Train loss 1.1984911487029903 accuracy 0.8210952380952381
====================Epoch 2====================
Train loss 0.7982195072805082 accuracy 0.8489047619047619
====================Epoch 3====================
Train loss 0.6282863611937494 accuracy 0.8644285714285714
====================Epoch 4====================
Train loss 0.5398486779822144 accuracy 0.8748571428571429
====================Epoch 5====================
Train loss 0.48565026496994673 accuracy 0.8818571428571429
====================Epoch 6====================
Train loss 0.44882184756381144 accuracy 0.888
====================Epoch 7====================
Train loss 0.4219776603309804 accuracy 0.8913809523809524
====================Epoch 8====================
Train loss 0.4013803538012503 accuracy 0.8938095238095238
====================Epoch 9====================
Train loss 0.38494411358043623 accuracy 0.8962380952380953
====================Epoch 10====================
Train loss 0.3713991677115017 accuracy 0.9
====================Epoch 11====================
Train loss 0.35993509069984364 accuracy 0.9026190476190477
====================Epoch 12====================
Train loss 0.35001411678101124 accuracy 0.9044285714285715
====================Epoch 13====================
Train loss 0.34129188324526655 accuracy 0.9062380952380953
====================Epoch 14====================
Train loss 0.3335065512064562 accuracy 0.9084285714285715
====================Epoch 15====================
Train loss 0.3264656930628604 accuracy 0.910047619047619
====================Epoch 16====================
Train loss 0.32003152913274996 accuracy 0.911952380952381
====================Epoch 17====================
Train loss 0.3140958608903743 accuracy 0.9137142857142857
====================Epoch 18====================
Train loss 0.3085850416110963 accuracy 0.9147142857142857
====================Epoch 19====================
Train loss 0.30343134056736204 accuracy 0.9162380952380952
====================Epoch 20====================
Train loss 0.29856572329647985 accuracy 0.9168571428571428
====================Epoch 21====================
Train loss 0.29394513709711 accuracy 0.918047619047619
====================Epoch 22====================
Train loss 0.2895373369746862 accuracy 0.9195714285714286
====================Epoch 23====================
Train loss 0.28532179326236035 accuracy 0.9208095238095239
====================Epoch 24====================
Train loss 0.2812839390251145 accuracy 0.9220952380952381
====================Training finished====================
```

```
Test loss 0.2968025462886123 accuracy 0.9164285714285715
```

Compare the results

In [43]:
```
import matplotlib.pyplot as plt

# Plot the training loss and accuracy separately.
plt.plot(range(n_epochs), train_loss_list, label='original train loss')
plt.plot(range(n_epochs), part_4_train_loss_list, label='new train loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.title('Training loss')
plt.legend()
plt.show()

plt.plot(range(n_epochs), train_accuracy_list, label='original train accuracy'
plt.plot(range(n_epochs), part_4_train_accuracy_list, label='new train accurac
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.title('Training accuracy')
plt.legend()
plt.show()
```





In [44]:

```
# plot sample test images together with their groundtruth
# and old and new predicted labels

fig, axes = plt.subplots(10, 10, figsize=(10, 10))
fig.subplots_adjust(wspace=0)
for a, image, true_label, pred_trained, new_pred_trained in zip(
        axes.flatten(), test_data[selected],
        true_labels, preds_trained, part_4_preds_trained):
    a.imshow(image.reshape(28, 28), cmap='gray_r')
    a.text(0, 10, str(true_label), color="black", size=15)
    a.text(0, 26, str(new_pred_trained), color="blue", size=15)
    a.text(22, 26, str(pred_trained), color="red", size=15)

    a.set_xticks(())
    a.set_yticks(())

plt.show()
```

Analyze your findings

How did different hyperparameters and network configurations affect the model training? Did you notice any patterns, and what was the best set of hyperparameters you tested? What other interesting observations did you see?

I tried a few different combinations. First, I tried keeping sigmoid as the activation function, and just increasing the number of hidden units from 16 to 256 (16^2). This resulted in around 0.5 loss and 0.8732448979591837 accuracy.

Next, I tried changing sigmoid to relu while keeping 256 hidden units. We talked about the pros of relu in class and also that it tends to perform better than sigmoid

on some tasks, at least better than sigmoid because it doesn't run into the vanishing gradient problem. With relu and 256 hidden units, we achieved:

- Test loss 0.29442339825935765 accuracy 0.9170204081632654

Then, keeping relu, I wanted to see if I had overshot the hidden units and instead tried both 128, and 192 with relu.

- 128 + relu, Test loss 0.3039259922261876 accuracy 0.9140204081632654
- 192 + relu, Test loss 0.3012087591625009 accuracy 0.9144897959183673

Reminder:

- 256 + relu, Test loss 0.29442339825935765 accuracy 0.9170204081632654

In the opposite direction, I tried 512 and 1024 hidden units with relu.

- 512 + relu: Test loss 0.2916698891047209 accuracy 0.918
- 1024 + relu : Test loss 0.28450889718456973 accuracy 0.9198979591836735

Finally, I switched out relu for tanh and tried all hidden unit combos

- 128 + tanh: Test loss 0.3191153522188792 accuracy 0.9094285714285715
- 192 + tanh: Test loss 0.32035236498481523 accuracy 0.9092244897959184
- 256 + tanh: Test loss 0.3225377038472999 accuracy 0.9086530612244897
- 512 + tanh: Test loss 0.32249800014512103 accuracy 0.9082857142857143
- 1024 + tanh: Test loss 0.32106903290551886 accuracy 0.909469387755102

Interestingly enough, for tanh, the most comparable results were between 128 hiden units and 1024 hiddne units, the ones with the largest discrepancy between them. Still, across all trials for tanh, the difference in accuracy and test loss was very small and not very significant. This is maybe to say that while more hidden units is better, sometimes the advantage is not that large, and a smaller amount of hidden units may be able to achieve the same result, faster.

With relu, the change as hidden units increased was a tiny bit more clear. As the # of hidden units increased, test loss decreased and accuracy increased, even if only by .1%. Therefore, my final choice would be relu with 256 units becuase it improved 0.25 from 192 + relu whereas all the others only improved ~ >= 0.1% from a trial with a smaller # of hidden units. Thus, 256 + relu seems like a sweet spot. There may be a slight plateau in improvement if we continue to increase the number of hidden units past 256.

Questions (10 pts):

1. Why are non-linear activation functions (like sigmoid or relu) necessary in an MLP? What would happen if you removed them and only used the linear transformation at each layer?

- If you remove the non-linear activation functions, then the neural network can be seen as a purely linear model like logistic regression. Essentially, your network is no different from a bunch of matrix multiplications and linear transformations, which theoretically could be written as just one big linear transformation. MLP would no longer be a multi-layer model because all of its operations could potentially be condensed into one layer. Overall, non-linear functions allow MLP models to learn non-linear/complex relationships between the data, since not all data can be represented well with linear models (the swiss roll example from class).

1. In the final plot of predictions in Task 3, find an image that the model still got wrong after training. Why do you think the model failed on this specific digit?

- Third column in, fourth row down, my model misclassifies an 8 as a 5. Just looking at the image, I can visualize a 5 over the existing lines in the image, especially because of the angle of the top right of the 8 and the whitespace in that top right that is characteristic of the top bar and vertical back of a 5. Futhermore, a 5 has a rounded bottom and this 8 doesn't look too dissimilar from the 5 in the second row, 7th column. Considering the other 8's in the plot, I think the orientation of the 8 (it's slant) is probably the main reason the number was misclassified. Many of the other 8s have a more vertically stacked circle or 90 90-degree rotated infinity sign look to them. Handwriting style can definitely make certain numbers look like other numbers and sometimes, make it even harder for a real human to distinguish one number from another, so it is

not inconceivable that the model is always going to get parts of this recognition task wrong!

1. The Layer class initializes weights using `np.random.uniform(-np.sqrt(2.0 / shape[0]), ...)`. Why can't we just initialize all weights and biases to zero? What problem would this cause during training?

- We can't initialize all weights and biases to zero (or even any same constant value for all of them) because then every hidden unit will receive the same input and produce the same output, which will lead to the same gradient update during backpropagation. At that point, every neuron will change in the exact same way and produce identical results - so then there would be no point to having so many neurons! As I discussed above, having many hidden units was beneficial, but in the case where each hidden layer does the same thing, the model cannot learn complexity very well and essentially acts as a bunch of neurons doing identical things. At that point, you could get the same result, likely with a singular neuron (but maybe that's an oversimplification). Random initializations make sure that the model can learn different features and patterns that underlie the data, not the same pattern across all neurons.